Building a Simple ETL Pipeline

Course: Data Engineering and Cloud Computing

Group members: Bwire Jude Wandera (B35390) · Geoffrey Oyet (B35603)

Date: 16 October 2026

1. ETL process implemented

The pipeline is a single Python script (`etl_pipeline.py`) that runs three stages in order. A stage starts only if the previous one succeeded.

Extract. pandas reads `sales.csv` (105 rows from three branches) with every column as text. We then convert each column ourselves, so we can see and handle bad values instead of letting pandas coerce them.

Transform. The cleaning steps, in the order they run:

Step	Rule applied	Rows affected
Trim whitespace	Leading/trailing spaces removed from all fields	All rows checked
Duplicates	Exact duplicate rows dropped, then any repeated `transaction_id` (first kept)	1 (`T010` appeared twice)
Missing values	Rows missing `quantity`, `unit_price`, `transaction_id` or `transaction_date` dropped; missing `branch` / `product` / `category` would be filled with `"Unknown"`	1 (`T028`, no quantity)
Date format	Dates converted to ISO `YYYY-MM-DD` using four explicit formats; rows whose date cannot be parsed are dropped	4 dates fixed (`T002`, `T003`, `T004`, `T008`), 0 dropped
Negative quantity	Rows with `quantity <= 0` removed	1 (`T101`, quantity −2)
New column	`total_amount = quantity × unit_price`	All rows

This leaves 102 clean rows. A validation step then checks that IDs are unique, no values are missing, all quantities are positive, every date is in ISO format and `total_amount` is correct. If any check fails, the pipeline stops before loading. The result is written to `cleaned_sales.csv`.

Load. The script moves the cleaned file into the simulated bucket at `cloud_storage_bucket/sales/2026/cleaned_sales.csv`, then loads it into a SQLite table called `sales`. The table schema (`sales_analytics.sql`) has a primary key on `transaction_id` and `CHECK` constraints, so the database also rejects bad rows. The script runs the four analytics queries and exports each result to `query_outputs/`.

Query results (UGX)

Branch	Transactions	Total revenue
Kampala	34	61,395,000
Mbarara	34	59,410,000
Jinja	34	51,435,000

Category	Transactions	Total revenue
Electronics	45	109,850,000
Furniture	29	42,170,000
Accessories	28	20,220,000

Top 5 products	Units sold	Total sales
Laptop	21	52,500,000
Monitor	43	36,550,000
Office Chair	67	23,450,000
Printer	32	20,800,000
Desk	39	18,720,000

Average transaction value: UGX 1,688,627.45 across 102 transactions. Total revenue is UGX 172,240,000, and both the branch totals and the category totals add up to that figure.

2. Challenges encountered

- **Ambiguous dates.** The file mixes `2026-01-03`, `2026/01/04`, `03/01/2026` and `05-01-2026`. A value like `03/01/2026` could mean 3 January or 1 March. We did not let pandas guess the format, because with day-first guessing turned on it can also misread the ISO dates. The IDs and dates run in sequence, so `T002` has to fall between 3 and 4 January. From this we concluded that the non-ISO formats are day-first, and we listed the accepted formats explicitly.
- **Missing values.** We could have filled `T028`'s missing quantity with an average or the most common value, but that would invent a sale and inflate revenue. Quantity is needed to compute `total_amount`, so we dropped the row. A missing category or product name does not change the amount, so those fields would be labelled `"Unknown"` instead.
- **Duplicates that look different.** `T010` was an exact copy. A resubmitted transaction could also arrive with a small change such as extra spaces, so we trim whitespace first and then deduplicate on `transaction_id`, the business key.
- **Order of cleaning steps.** Quantity has to be converted to a number before missing and negative values can be checked. We deduplicate before dropping any other rows, so each count in the log matches one problem.

3. How this simulates a real cloud data pipeline

This project	Real cloud equivalent
`sales.csv` dropped by branches	Files landing in a raw/landing zone (e.g. `s3://company-raw/`)
`etl_pipeline.py`	A scheduled job: AWS Glue, Azure Data Factory, Cloud Dataflow, or an Airflow task
`cloud_storage_bucket/`	An object storage bucket (Amazon S3, Azure Blob Storage, Google Cloud Storage)
`sales/2026/cleaned_sales.csv`	An object key with prefixes used for partitioning
SQLite `sales` table	A cloud data warehouse (BigQuery, Redshift, Snowflake, Synapse)
`query_outputs/*.csv`	Reports and dashboards (Power BI, Looker, QuickSight)

Object storage concepts. A bucket holds a flat set of objects with no real folders. Each object is found by a key such as `sales/2026/cleaned_sales.csv`. The `/` is a naming convention, and cloud consoles display it as folders. Organising keys as `dataset/year/` is called partitioning. Analytics engines can then read only one year's data, and storage rules (for example "move objects older than 2 years to cheaper archive storage") can apply to a single prefix. Objects are written whole and never edited in place, so re-running the pipeline replaces the object the same way an S3 `PUT` would. We keep the raw `sales.csv` unchanged and write the cleaned data to a separate location, which follows the common raw → curated "zone" pattern and means the cleaning can always be re-run from the original data.

What a production version would add: scheduling (e.g. a daily trigger when a branch uploads a file), a date partition per load (`sales/2026/01/15/`), a columnar format such as Parquet instead of CSV, alerts when validation fails, and IAM permissions on the bucket.